

Lab 0. QEMU Simulator and Raspberry Pi Setup

Revised edition — package names updated for current Ubuntu releases

We set up a simulation environment with QEMU to run Raspberry Pi code on your PC. The following instructions are for Windows 11.

Step 1. Set up QEMU

The video below is a useful general introduction to QEMU on Windows. Watch it for context, but install the packages listed in this handout rather than the ones in the video, and skip the last step of installing Ubuntu Linux.

[QEMU Installation Guide for Windows PC \[with KVM\]](#)

1.1 Enable the Windows features

From the Search box, start "Turn Windows features on or off" and enable:

- Windows Subsystem for Linux (required)
- Virtual Machine Platform (required)
- Hyper-V (optional for this lab — see the note below)

Note: This lab emulates an ARM machine on an x86 PC, which does not use hardware virtualization at all. Hyper-V is therefore not required. If you are running Windows 11 Home you will not see a Hyper-V entry in this list; that is expected, and WSL 2 still works.

1.2 Install WSL

From Windows PowerShell, run:

```
ws1 --install
ws1 --set-default-version 2
```

When starting WSL, always choose "run as administrator".

1.3 Install QEMU inside WSL

From WSL, run:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y qemu-system-arm qemu-utils
```

Verify the installation:

```
qemu-system-aarch64 --version
```

Note: Emulating a Raspberry Pi on an x86 PC is cross-architecture emulation. QEMU translates ARM instructions in software using its TCG engine, so KVM, Hyper-V and /dev/kvm play no role here. The emulated Pi will boot noticeably more slowly than real hardware — this is normal, and the first boot can take several minutes.

1.4 If you also want to run x86 virtual machines (optional)

Older versions of this handout, and the video above, tell you to run:

```
sudo apt install qemu-kvm libvirt-daemon-system libvirt-clients bridge-utils -y
```

On current Ubuntu releases this command fails, because qemu-kvm is now a virtual package with no installable candidate. KVM support has not been removed — it is provided by qemu-system-x86. None of these packages

are needed for this lab. If you want them anyway, install:

```
sudo apt install -y qemu-system-x86 libvirt-daemon-system libvirt-clients bridge-utils
```

Hardware acceleration only works if Linux exposes the KVM device. Check with:

```
ls -l /dev/kvm
```

If that file exists, add yourself to the kvm group and restart WSL:

```
sudo usermod -aG kvm $USER
# then, from Windows PowerShell:
wsl --shutdown
```

If /dev/kvm does not exist (common under WSL), skip the usermod step entirely and let QEMU fall back to software emulation with -accel tcg.

1.5 Mac users

Please refer to the following links:

- Install QEMU on OSX: <https://gist.github.com/Jatapiaro/6a7c769a07911adc629e1604729d4c7a>
- How To Install And Use QEMU Virtual Machine On macOS With HVF Acceleration: <https://www.youtube.com/watch?v=EZ56AdVO2Mc>
- How to install Linux on Mac step by step: <https://macpaw.com/how-to/install-linux-on-mac>

If you run into difficulties on Windows or Mac, I can give you a Raspberry Pi board. Note that SSH to Linux machines in the lab does not work, since you need root privileges to install QEMU on the machine.

Step 2. Install Raspberry Pi OS on QEMU

This step is based on this page for installing Raspberry Pi on QEMU: [Emulating a Raspberry Pi in QEMU](#)

However, we need more storage in the emulated Raspberry Pi, so please follow the updated instructions below.

1. Get the Raspberry Pi image

QEMU was already installed in Step 1.3. If you skipped it, install it now — note that the binary is called qemu-system-aarch64, but the package that provides it is qemu-system-arm:

```
sudo apt install -y qemu-system-arm
cd ~
wget https://downloads.raspberrypi.org/raspios_arm64/images/raspios_arm64-2023-05-03/2023-05-03-raspios-bullseye-arm64.img.xz
xz -d 2023-05-03-raspios-bullseye-arm64.img.xz
```

2. Inspect and mount the boot partition

```
fdisk -l ./2023-05-03-raspios-bullseye-arm64.img
# Note the boot partition offset (should be 8192 * 512 = 4194304)
sudo mkdir /mnt/image
sudo mount -o loop,offset=4194304 ./2023-05-03-raspios-bullseye-arm64.img /mnt/image/
```

Extract the kernel and device tree:

```
cp /mnt/image/bcm2710-rpi-3-b-plus.dtb ~
cp /mnt/image/kernel8.img ~
```

3. Resize the image to 32 GB (or your desired size)

Instead of resizing to the next power of two, pick a larger number directly. For example, 32 GB:

```
qemu-img resize ./2023-05-03-raspbios-bullseye-arm64.img 32G
```

You can change 32G to 16G, 64G, etc.

4. Enable SSH

Create a hashed password and add userconf + ssh. In the echo command, replace <your_hash_here> with what you got from running "openssl passwd -6". Copy this command into a text file and edit it there before pasting it back.

```
openssl passwd -6
# type your password twice; copy the hash
echo 'pi:<your_hash_here>' | sudo tee /mnt/image/userconf
sudo touch /mnt/image/ssh
```

Unmount:

```
sudo umount /mnt/image
```

5. Launch QEMU with the expanded disk

```
qemu-system-aarch64 \
-machine raspi3b -cpu cortex-a72 -nographic \
-dtb bcm2710-rpi-3-b-plus.dtb \
-m 1G -smp 4 \
-kernel kernel8.img \
-sd 2023-05-03-raspbios-bullseye-arm64.img \
-append "rw earlyprintk loglevel=8 console=ttyAMA0,115200 dwc_otg.lpm_enable=0 \
root=/dev/mmcblk0p2 rootdelay=1" \
-device usb-net,netdev=net0 -netdev user,id=net0,hostfwd=tcp::5010-:22
```

You can log into the Pi within the current window, or you can SSH in from another WSL window:

```
ssh -p 5010 pi@localhost
```

Save the above commands in a text file (for example start-pi.sh) for starting up the simulator in the future.

Note: Do not add -enable-kvm or -accel kvm to this command. The guest is ARM and your PC is x86, so QEMU must use software translation. Booting to the login prompt may take several minutes.

6. Expand the filesystem inside the emulated Pi

Once inside the Pi:

```
sudo raspi-config
```

- Go to Advanced Options -> Expand Filesystem
- Reboot

Alternatively (manual method, usually not needed):

```
sudo fdisk /dev/mmcblk0
# delete partition 2 and recreate it to extend to the full disk
sudo reboot
sudo resize2fs /dev/mmcblk0p2
```

7. Verify the new size

```
df -h
```

You should now see nearly 32 GB available in the root folder /.

This modified workflow only changes Step 3 (resize) and Step 6 (expand FS); everything else is identical to the

original blog post's process.

Step 3. Enable 32-bit ARMv7 assembly on Raspberry Pi 3/4

Follow the next steps for running 32-bit ARMv7 assembly programs on a Raspberry Pi 3/4 running Raspberry Pi OS 64-bit.

1. Verify hardware and OS architecture

```
uname -m          # should show aarch64
lscpu | grep -i model
```

Raspberry Pi 3 and 4 (ARMv8-A) can run ARMv7 binaries if the kernel supports AArch32.

2. Enable 32-bit (armhf) support in Raspberry Pi OS 64-bit

By default the 64-bit OS is multi-arch capable. You need the 32-bit runtime:

```
sudo dpkg --add-architecture armhf
sudo apt update
sudo apt install libc6:armhf libstdc++6:armhf
```

You also need the 32-bit assembler and linker used in the next section:

```
sudo apt install binutils-arm-linux-gnueabi
```

If you want to compile C programs for ARMv7 as well (not required for this lab):

```
sudo apt install gcc-arm-linux-gnueabi g++-arm-linux-gnueabi
```

Note: Earlier versions of this handout listed `binutils-arm-linux-gnueabi` as optional. It is not — without it the `arm-linux-gnueabi-as` and `arm-linux-gnueabi-ld` commands below do not exist.

3. Generating an ARMv7 binary

Save your ARMv7 assembly program, e.g. `hello.s`, using a text editor (see the `vi` tutorial). The program is from this page: <https://embeddedjourneys.com/blog/hello-world-arm-assembly-raspberry-pi/>

(Note: this program does not work in CPULator, since CPULator does not implement Linux system calls via `SWI 0`.)

```
.global _start
_start:
    MOV    R0, #1
    LDR    R1, =msg
    MOV    R2, #12
    MOV    R7, #4
    SWI    0
    MOV    R7, #1
    SWI    0
msg:
    .asciz "Hello, ARM!\n"
```

Note: The directive is `.global` with a leading dot. Writing `global _start` without the dot is a common source of "undefined reference" errors at link time.

Then assemble and link with the 32-bit toolchain:

```
arm-linux-gnueabi-as -o hello.o hello.s
arm-linux-gnueabi-ld -o hello hello.o
```

You will notice the output binary is ELF32 for ARM, which runs in AArch32 compatibility mode. You can create command aliases to reduce typing, c.f. *How to Create Aliases (Shortcuts) for Common Commands in Linux*. For

example, if you are running the bash shell, edit `.bashrc`:

```
vi ~/.bashrc
```

and add these lines:

```
alias as32='arm-linux-gnueabi-hf-as'
alias ld32='arm-linux-gnueabi-hf-ld'
alias gcc32='arm-linux-gnueabi-hf-gcc'
```

Note: Use plain ASCII straight quotes in `.bashrc`. Curly quotes copied from a PDF or word processor will produce a syntax error.

Check with:

```
file hello
# ./hello: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), statically linked, not
stripped
```

4. Run the ARMv7 program directly

```
./hello
```

The Pi's 64-bit kernel automatically switches into AArch32 user mode for execution.

5. Debug / inspect (optional)

You can disassemble or inspect with:

```
arm-linux-gnueabi-hf-objdump -d hello
```

Step 4. Lab Report

Write a short report that records your experience in going through these steps. What problems did you encounter, and how did you resolve them?

Changes in this revision

- Step 1: `qemu-kvm` replaced with `qemu-system-arm` and `qemu-utils`. On current Ubuntu releases `qemu-kvm` is a virtual package and the old command fails with "has no installation candidate".
- Step 1: `libvirt`, `bridge-utils` and the `kvm` group membership moved to an optional section, since cross-architecture emulation of the Pi cannot use KVM.
- Step 1: Hyper-V marked optional, with a note for Windows 11 Home users.
- Step 2.1: package name corrected from `qemu-system-aarch64` (not a package) to `qemu-system-arm`, which provides the `qemu-system-aarch64` binary.
- Step 2: duplicated "8. Verify the New Size" section removed.
- Step 3.2: `binutils-arm-linux-gnueabi-hf` moved from optional to required.
- Step 3.3: `.global` directive corrected (leading dot) and the listing indented.